SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

---

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets
Assessment Tool Weightage: 35%

QUESTIONS: 05 Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding *(2)*	Algorithm Design *(3)*	Use of Control Structures *(2)*	Pseudocode Structure *(1)*	Completeness *(2)*	***Total(10)***
Weightage	*20%*	30%	20%	10%	20%	*100%*
Q1						

Q2						
Q3						
Q4						
Q5						

Code of

Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action. ***Signature***

of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightag e	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understandin g	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencie s	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear

		follow			
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

SIMATS ENGINEERING

Assessment Tool 2 — Pseudocode Writing Task (Algorithm Design)

Association Rule Mining — Pseudocode Answer Key

CSA16 · Data Warehousing and Data Mining • CO5

Course	CSA16 — Data Warehousing & Data Mining	**CO5**	Association rule mining
Questions	05	**Total**	50 Marks · Weightage 35%

Each answer states inputs/outputs/constraints, gives structured pseudocode (the main deliverable), works a short illustration on the supplied dataset where the question asks for it, and notes the control structures and cases handled — matching the rubric (Problem Understanding, Algorithm Design, Use of Control Structures, Pseudocode Structure, Completeness).

Illustrations assume minimum support = 40% (count ≥ 2 of 5) and, where rules are generated, minimum confidence = 70%, unless the question fixes a value.

Problem understanding — inputs / outputs / constraints

- **Input:** transaction set T (online orders), minimum support threshold
- **Output:** all frequent itemsets (product combinations)
- **Requirement:** show candidate generation + pruning at each iteration

Pseudocode — Apriori with join & prune

```
FUNCTION Apriori(T, min_sup):
 L[1] <- { {i} : item i in T AND support({i},T) >= min_sup }
 k <- 2
 WHILE L[k-1] is not empty DO
 C[k] <- Apriori_Gen(L[k-1]) // generate candidates
 FOR EACH transaction t IN T DO // count supports
 FOR EACH candidate c IN C[k] DO
 IF c SUBSET_OF t THEN c.count <- c.count + 1
 L[k] <- { c IN C[k] : c.count/|T| >= min_sup } // prune by support
 k <- k + 1
 RETURN UNION over all L[i]

FUNCTION Apriori_Gen(L_prev):
 C <- {}
 FOR EACH pair p,q IN L_prev WITH same first (k-2) items DO
 cand <- p UNION q // join step
 IF NOT Has_Infrequent_Subset(cand, L_prev) THEN
 ADD cand TO C // prune step
 RETURN C

FUNCTION Has_Infrequent_Subset(cand, L_prev):
 FOR EACH (k-1)-subset s OF cand DO
 IF s NOT IN L_prev THEN RETURN TRUE // downward closure
 RETURN FALSE
```

Worked illustration on the given dataset

Orders: O1{Mobile,Earphones}, O2{Mobile,Charger}, O3{Earphones,Charger}, O4{Mobile,Earphones,Charger}, O5{Mobile,Earphones}.

L1: Mobile=4, Earphones=4, Charger=3 (all ≥ 2).

L2: {Mobile,Earphones}=3, {Mobile,Charger}=2, {Earphones,Charger}=2 → all frequent. C3: {Mobile,Earphones,Charger}=1 (O4 only) → pruned. Mining stops.

Problem understanding — inputs / outputs / constraints

- **Input:** transaction set T (book borrows), minimum support
- **Output:** frequent patterns, found without candidate generation
- **Requirement:** build an FP-Tree from the records and mine it

Pseudocode — FP-Growth (build tree + recursive mine)

```
FUNCTION FP_Growth(T, min_sup):
 F <- frequent items in T (support >= min_sup), sorted by count DESC
 tree <- new FPTree(root=NULL); header <- {}
 FOR EACH transaction t IN T DO
 ordered <- items of t that are in F, in F's order
 Insert_Tree(ordered, tree.root, header)
 RETURN FP_Mine(header, {}, min_sup)

FUNCTION Insert_Tree(items, node, header):
 IF items is empty THEN RETURN
 x <- items.head
 IF node has child ch WITH ch.item = x THEN ch.count <- ch.count + 1
 ELSE
 ch <- new Node(x, count=1, parent=node)
 LINK ch INTO header[x] list
 Insert_Tree(items.tail, ch, header) // recurse on rest

FUNCTION FP_Mine(header, suffix, min_sup):
 patterns <- {}
 FOR EACH item i IN header (ascending count) DO
 p <- suffix UNION {i}; ADD p TO patterns
 base <- conditional pattern base of i // prefix paths + counts
 condTree, condHeader <- Build_Tree(base, min_sup)
 IF condTree not empty THEN
 patterns <- patterns UNION FP_Mine(condHeader, p, min_sup)
 RETURN patterns
```

Worked illustration on the given dataset

Counts: Python=4, ML=4, DataScience=3 (all ≥ 2) → order [Python, ML, DataScience].
Ordered rows: B1[Py,DS], B2[Py,ML], B3[ML,DS], B4[Py,ML,DS], B5[Py,ML].
Insert each row into the tree, incrementing shared-prefix node counts (Py branch is shared by B1,B2,B4,B5).
Mine bottom-up from the header table (DS → ML → Py) using conditional pattern bases — no candidate itemsets are ever generated.

Problem understanding — inputs / outputs / constraints

- **Input:** frequent itemsets with support counts, minimum confidence threshold
- **Output:** association rules passing the confidence filter
- **Requirement:** confidence calculation, rule generation, filtering

Pseudocode — Generate_Rules from frequent itemsets

```
FUNCTION Generate_Rules(F, support[], min_conf):
 Rules <- {}
 FOR EACH itemset f IN F WITH |f| >= 2 DO
 FOR EACH non-empty proper subset A OF f DO
 B <- f - A // consequent
 conf <- support[f] / support[A] // confidence
 IF conf >= min_conf THEN
 ADD (A -> B, support[f], conf) TO Rules
 RETURN Rules
```

Worked illustration on the given dataset

Given counts: {Internet}=4, {Streaming}=4, {Cloud}=3, {Internet,Streaming}=3, {Streaming,Cloud}=2. min_conf = 70%.

From {Internet,Streaming}=3: Internet→Streaming = 3/4 = 75% ✓ , Streaming→Internet = 3/4 = 75% ✓ From {Streaming,Cloud}=2: Streaming→Cloud = 2/4 = 50% ✗ , Cloud→Streaming = 2/3 = 66.7% ✗ **Valid rules:** Internet→Streaming and Streaming→Internet (both 75%).

Problem understanding — inputs / outputs / constraints

- **Input:** transactions T (course enrolments), min support, anchor item = Python
- **Output:** frequent itemsets that all contain Python
- **Requirement:** explain how the constraint affects candidate generation & pruning

Pseudocode — anchored constrained frequent-itemset mining

```
FUNCTION Constrained_FIM(T, min_sup, anchor):
 IF support({anchor}, T) < min_sup THEN RETURN {} // anchor infrequent
 // keep only items that are frequent TOGETHER WITH the anchor
 F <- { i : i != anchor AND support({i,anchor},T) >= min_sup }
 result <- { {anchor} }
 base <- { {i} : i IN F }
 WHILE base is not empty DO
 next <- {}
 FOR EACH s IN base DO
 cand <- s UNION {anchor} // every set keeps anchor
 IF support(cand, T) >= min_sup THEN
 ADD cand TO result
 ADD s TO next
 base <- Apriori_Gen(next) // grow non-anchor part
 RETURN result
```

Worked illustration on the given dataset

Enrolments: S1{Py,DB}, S2{Py,AI}, S3{DB,AI}, S4{Py,DB,AI}, S5{Py}. support(Py)=4 ✓. Items frequent with Python: {Py,DB}=2 ✓, {Py,AI}=2 ✓.

Grow: {Py,DB,AI}=1 (S4 only) ✗. **Result:** {Python}, {Python,DB}, {Python,AI}.

Constraint effect: candidates without Python are never generated, so S3{DB,AI} and pair {DB,AI} are pruned immediately — a large cut in the search space.

Problem understanding — inputs / outputs / constraints

- **Input:** transactions T, a concept hierarchy H, per-level min support, min confidence ■ **Output:** rules at each hierarchy level
- **Requirement:** explain rule generation across levels and the value of concept hierarchies

Pseudocode — level-by-level multilevel mining

```
FUNCTION Multilevel_ARM(T, H, min_sup[], min_conf):
 Rules <- {}
 FOR level = TOP DOWNTO BOTTOM OF H DO
 // re-encode each item as its ancestor concept at this level
 T_l <- { map(item -> ancestor_at(level,H)) for each t IN T }
 F_l <- Apriori(T_l, min_sup[level]) // level-specific support
 R_l <- Generate_Rules(F_l, support_l, min_conf)
 Rules <- Rules UNION R_l
 RETURN Rules
 // NOTE: use smaller min_sup[level] at deeper levels (support shrinks)
```

Worked illustration on the given dataset

Hierarchy: Healthcare → {Cardiology → {ECG}, Orthopedics}.
Level 1 Healthcare = 5/5 (100%) — general, near-trivial.
Level 2: Cardiology=3 (60%), Orthopedics=2 (40%). Level 3: ECG=2 (40%).
Low-level rule **ECG → Cardiology = 2/2 = 100%** (and ECG → Healthcare 100%).
Concept hierarchies let general patterns surface high up (high support) and specific ones lower down (with reduced support), so useful rules are not missed at a single level.